DeepFakeFace – Deep Learning Project Report

Daniel Dias, Lucas Santiago, Rafael Conceição
(Dated: April 7, 2025)

This project explores the application of deep learning techniques to the problem of deepfake detection and generation. Using the DeepFakeFace dataset (DeepFakeFace), we developed two types of models: discriminative models to classify images as real or fake, and generative models to synthesize new fake images. The discriminative model was trained using convolutional neural networks, visual transformers, and YOLO (Ultralytics) while the generative model leveraged DCGAN and StyleGAN architectures. Our results show promising classification performance and the ability to generate visually convincing fake images. The work also reflects iterative improvements based on model performance during training.

I. INTRODUCTION

In recent years, the emergence of deepfake technology—synthetically generated media content created using deep learning—has introduced both impressive innovations and serious ethical concerns. Deepfakes are primarily created using generative models such as Generative Adversarial Networks (GANs), diffusion models, and autoencoders, and are capable of producing highly realistic fake images or videos that are increasingly difficult to detect.

While these technologies have legitimate applications in entertainment, gaming, and accessibility, they also pose a risk in terms of misinformation, fraud, identity theft, and political manipulation. As such, the ability to both generate and detect deepfake content has become a central challenge in the field of computer vision and artificial intelligence.

This project addresses both sides of this challenge:

- **Discriminative task**: designing and training a deep learning model capable of detecting whether an image is real or artificially generated.
- **Generative task**: building a model capable of generating fake images that resemble real human faces convincingly.

To achieve these goals, we use the DeepFakeFace dataset, which provides a large collection of real and fake face images. The fake images in this dataset were generated using three different deep learning-based tools: Stable Diffusion v1.5, Stable Diffusion Inpainting, and InsightFace.

Beyond implementation, the core goal of this project is to iteratively improve the proposed models based on experimental observations, hyperparameter tuning, and validation feedback. The project offers hands-on experience with deep learning workflows, from data preprocessing and model design to evaluation and result interpretation.

II. DATASET

We used the DeepFakeFace (DFF) dataset, which includes:

- **30,000 real images** from the IMDB-WIKI dataset.
- **90,000 fake images** generated using:
  - Stable Diffusion v1.5
  - Stable Diffusion Inpainting
  - InsightFace

The final dataset we used for the training of the models used all the 30,000 real images and 10,000 images of each fake generator. The dataset is balanced and suitable for both classification and generation tasks.

A. Classification Dataset

For the classification task, no major preprocessing was done, the images were just resized accordingly to the model used. A resized / reshaped fake image is still a fake image, vice versa.

B. Generation Dataset

During the generation phase, we observed that images often included excessive background noise, capturing more than just the faces. Simply resizing images was inadequate, as it led to the generator learning from distorted, stretched images. Central cropping was also insufficient, as it risked omitting facial features when faces were not centered.

For the generation task, images were preprocessed using different cropping strategies to reduce background noise and focus on faces. Central cropping alone was insufficient, so we used face detection with OpenCV and MediaPipe to get better training inputs. Dlib was also tested and ended up being the method used even though its slow iteration.

To address these issues, we developed methods to crop images more effectively, ensuring higher quality inputs for the generator and thereby improving the generation results.

We tried different methods and compared their accuracy about detecting faces and applying the 'face crop':

- `create_30k_real_central_crop.py`
  This script performs central cropping on images to create a dataset of 30,000 images. However, central cropping may inadvertently exclude important facial features if the face is not perfectly centered.
- `create_30k_real_face_crop_cv2.py`
  Utilizes OpenCV (`cv2`) for face detection and cropping. OpenCV's face detection is generally fast but may not be as accurate as other methods, leading to a certain percentage of unsuccessful crops.
- `create_30k_real_face_crop_dlib.py` (chosen method)
  Employs `dlib` for face detection and cropping. While `dlib` offers accurate face detection, its approach requires longer processing time.
- `create_30k_real_face_crop_mediapipe.py`
  Leverages Google's MediaPipe for face detection and cropping. MediaPipe provides a balance between speed and accuracy, often yielding a higher percentage of successfully cropped images.

Method	% Face Cropped	% Center Cropped	Detection Method
OpenCV (cv2)	82.0%	18.0%	multi
MediaPipe	74.0%	26.0%	static
Dlib	89.0%	11.0%	–
Central Crop	0.0%	100.0%	–

TABLE I: Cropping performance across different face detection methods.

By implementing these tailored cropping methods, we ensured that the images fed into the generator were of higher quality, focusing on faces and minimizing background noise. This approach significantly enhanced the performance and realism of the generated deepfake outputs.

III. DISCRIMINATIVE MODELS

We implemented and compared multiple architectures for image classification:

- **CNN with ResNet**: A convolutional neural network utilizing residual connections to enable deeper architectures.
- **Vision Transformer (ViT)**: A transformer-based model operating on image patches for classification tasks.
- **YOLOv11**: Adapted for image classification tasks, YOLOv11 processes images at high frame rates, making it suitable for real-time detection scenarios.

For each model, we employed specific training strategies:

- **Input Size**: All models were trained with images resized to 224x224 pixels (in CNN and ViT) and 512x512 pixels (in YOLO) to maintain consistency across experiments.
- **Optimizer**: Adam optimizer with an initial low learning rate.
- **Loss Function**: CrossEntropyLoss for multi-class classification.
- **Data Split**: 80% training, 10% validation, 10% testing.
- **Batch Size**: 32.
- **Epochs**: 50.

We also developed an interesting research: **"How do pretrained models affect the performance of the classification?"**. This will be explored in the results section.

IV. GENERATIVE MODELS

A. Architectures Used

We implemented and tested two generative models:

- **DCGAN**: Deep Convolutional Generative Adversarial Network with transposed convolutions and batch normalization.
- **StyleGAN-like Architecture**: A custom architecture inspired by StyleGAN, incorporating progressive generation layers and style-based modulation to enhance image realism.

For the generative models, we adopted the following training strategies:

- **Input Size**: Images resized to 128x128 pixels for faster computing.
- **Latent Vector (Noise)**: 100-dimensional.
- **Optimizer**: Adam with a learning rate of 0.0002 for both generator and discriminator.
- **Loss Function**: Binary Cross Entropy.
- **Batch Size**: 64.
- **Epochs**: 250, monitored for convergence.

As mentioned in the Section II B we improved the generation task by simply cropping the important features of the images. Only this small change made a huge difference. This will be explored in the results section.

V. CLASSIFICATION RESULTS

We evaluated the performance of several deep learning models under two configurations:

- **Random initialization (no pretrained weights)**
- **Pretrained models (fine-tuned on our dataset)**

To evaluate the models, we measured the following metrics:

- **Accuracy**: Proportion of correct predictions
- **F1-Score**: Harmonic mean of precision and recall
- **Precision/Recall**: Per-class and macro-averaged values

Table II shows the performance of each classifier.

TABLE II: Classification performance with and without pretrained weights

Model	Pretrained	Accuracy	Comments
ResNet18	Yes	98.30%	Good performance
ResNet18	No	84.23%	Pretraining really helps model performance
ViT	Yes	92.95%	Good result overall but very resource consuming
ViT	No	71.32%	Bad performance without pretrained weights
ViT (no dropout)	Yes	84.46%	No dropout really worsens the accuracy
YOLO	Yes	98.99%	Best performing. Fast and reliable
YOLO	No	60.50%	Fast but bad accuracy. Needs more data for training

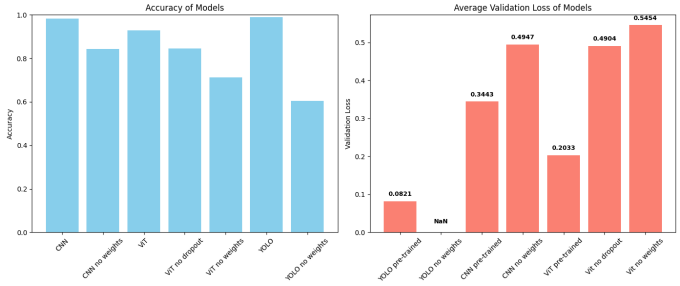

FIG. 1: Classifier Evaluation

Some observations we can take:

- Fine-tuning pretrained models consistently outperformed training from scratch.
- ViT achieved a good accuracy, although it required longer training. Could be better for the huge amount of computational resources it used.
- ResNet18 performed very well with pretrained weights and proved robust to different fake generation styles.
- YOLOv11 provided fast inference and good generalization. State-of-the-art algorithm for this type of tasks.

Finally we answered the question mentioned in Section III **"How do pretrained models affect the performance of the classification?"**. Yes, pretrained models really help the performance of the studied models and allow for stronger, generalized and robust accuracy ass seen in the following image:

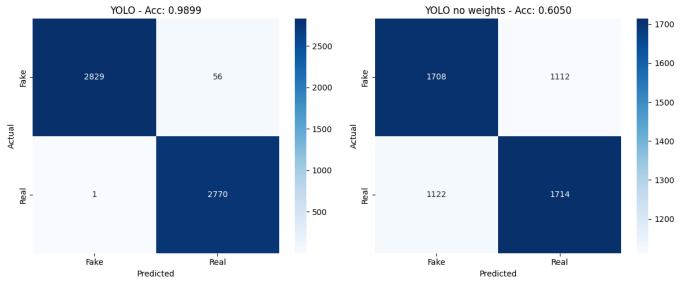

FIG. 2: YOLO pretrained vs No weights

More detailed logs, confusion matrices, and class-wise precision/recall scores are available in the provided benchmarking notebook: https://github.com/v2denny/deepfakeface/blob/main/classifier_benchmarking.ipynb.

VI. GENERATIVE RESULTS

In this section, we evaluate the performance of our generative models—DCGAN and the StyleGAN-like ar-

chitecture, using both qualitative and quantitative measures.

To assess the visual quality of the generated images, we generated sets of samples from both models. Figures 3 and 4 show grids of 16 generated images from DCGAN and the StyleGAN-like model, respectively. These grids were produced using fixed latent vectors for consistency, which facilitates a direct visual comparison. While the DCGAN outputs capture the overall structure of faces, the StyleGAN-like outputs exhibit finer details and improved clarity.

FIG. 3: Generated samples from the DCGAN model.

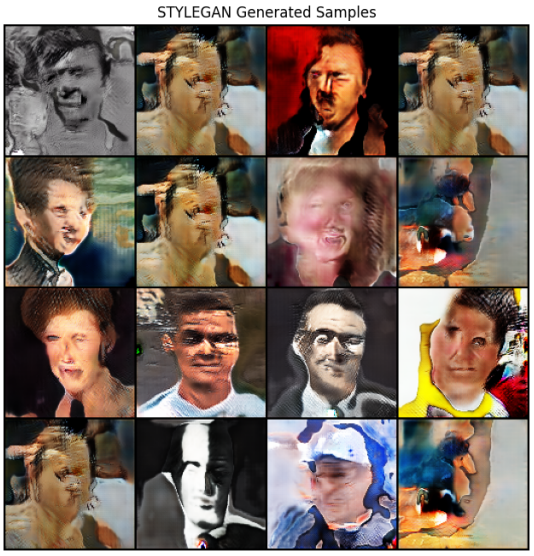

FIG. 4: Generated samples from the StyleGAN-like model.

To further quantify the quality of the generated images, we computed two metrics: the Frechet Inception Distance (FID) and the "fooling rate" (FR) (the number of times the generator can fool the discriminator). Lower FID values indicate that the distribution of generated images is closer to that of the real images, while higher FR values suggest greater realism.

Our evaluations yielded:

Model	Fooling Rate	FID
DCGAN	113/1000 (11.30%)	115.32
StyleGAN	44/1000 (4.40%)	192.86

TABLE III: Evaluation Metrics for DCGAN and StyleGAN Models

While there is room for further refinement, these results provide a promising baseline for our generative models. The DCGAN achieved a fooling rate of 11.30%, indicating that a significant portion of its generated images successfully falls within the ambiguity zone of the discriminator, suggesting that the model is learning realistic features. Similarly, the StyleGAN-like model's fooling rate of 4.40% shows that it is capturing critical facial characteristics, albeit with more conservative outputs.

The FID scores—115.32 for DCGAN and 192.86 for the StyleGAN-like model—are encouraging starting points given the complexity of generating photorealistic faces from scratch.

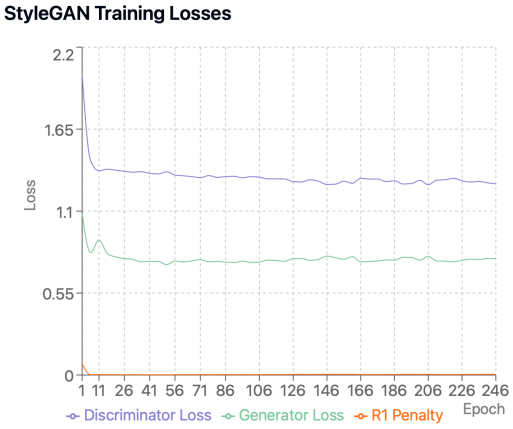

FIG. 5: Training loss curves for StyleGAN.

Figure 5 illustrates the generator and discriminator loss curves over the training period. StyleGAN demonstrates stabilization really early on while dcGAN only show stabilization after approximately 100 epochs. Both models demonstrate adversarial dynamics indicating a balanced competition between the generator and discriminator.

To further assess the quality of the generated images, we extracted features from real and generated images using a pretrained Inception network. We then applied t-SNE to visualize the high-dimensional feature distributions. Figure 6 shows that the features from dcGAN

generated images are closer and even overlap with those from the real images, suggesting that the generator has learned a feature representation close to that of the real data.

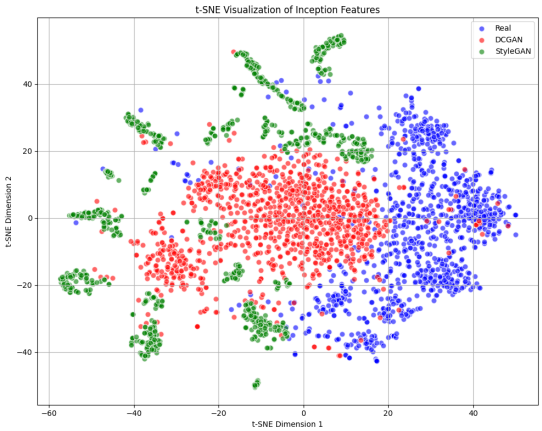

FIG. 6: t-SNE visualization of features extracted from real and generated images. Blue points represent real images, while red points denote generated images.

The evaluations show that while both models are capable of generating recognizable human faces, the StyleGAN-like model outperforms the DCGAN in terms of delails while DCGAN displays way better quantitative metrics. StyleGAN architecture produces images that are more similar to the real dataset. These improvements can be attributed to the style-based modulation and progressive generation layers, which allow finer control over the image synthesis process.

VII. CONCLUSION

This project demonstrated the effectiveness of deep learning approaches for both the detection and generation of deepfake images using the DeepFakeFace dataset. Our experiments with discriminative models—including CNNs with ResNet, Vision Transformers, and a YOLO-based architecture—clearly showed that leveraging pre-trained weights significantly improves performance, with YOLOv11 achieving near-perfect accuracy in detecting fake images. On the generative side, while DCGAN produced better quantitative metrics, the StyleGAN-like model yielded finer details and enhanced visual realism, underscoring the trade-offs between quantitative performance and perceptual quality. Furthermore, our investigation into image preprocessing techniques, particularly the use of tailored face cropping methods, played a crucial role in enhancing the overall quality of the inputs, thereby positively influencing both classification and generation outcomes.

VIII. FUTURE WORK

Future research will focus on several avenues to further advance the capabilities of deepfake detection and generation. First, exploring more advanced architectures and state-of-the-art data augmentation strategies may yield even more robust and generalized discriminative models. In addition, fine-tuning the balance between quantitative metrics and perceptual quality in generative models remains an open challenge; integrating improvements from recent advancements in GAN architectures could enhance the realism of generated images. Moreover, expanding the dataset with more diverse sources of both real and deepfake images, as well as incorporating temporal consistency for video deepfakes, will be essential for tackling evolving real-world scenarios. Finally, addressing the ethical implications and developing real-time detection systems are critical next steps to ensure the responsible use and monitoring of deepfake technology.